

A Practical Guide to Autoencoder

TOPIC -- AUTOENCODER

-- 01 --

Image processing The characteristics of autoencoders are useful in image processing. One example can be found in lossy image compression, where autoencoders outperformed other approaches and proved competitive against JPEG 2000. Another useful application of autoencoders in image preprocessing is image denoising. Autoencoders found use in more demanding contexts such as medical imaging where they have been used for image denoising as well as super-resolution. In image-assisted diagnosis, experiments have applied autoencoders for breast cancer detection and for modelling the relation between the cognitive decline of Alzheimer's disease and the latent features of an autoencoder trained with MRI.

-- 02 --

Variational autoencoders (VAEs) belong to the families of variational Bayesian methods. Despite the architectural similarities with basic autoencoders, VAEs are architected with different goals and have a different mathematical formulation. The latent space is, in this case, composed of a mixture of distributions instead of fixed vectors. Given an input dataset x characterized by an unknown probability function $P(x)$ and a multivariate latent encoding vector z , the objective is to model the data as a distribution $p_{\theta}(x)$, with θ defined as the set of the network parameters so that $p_{\theta}(x) = \int z p_{\theta}(x, z) dz$.

-- 03 --

Training an autoencoder An autoencoder, by itself, is simply a tuple of two functions. To judge its quality, we need a task. A task is defined by a reference probability distribution μ_{ref} over X , and a "reconstruction quality" function $d: X \times X \rightarrow [0, \infty]$, such that $d(x, x')$ measures how much x' differs from x . With those, we can define the loss function for the autoencoder as $L(\theta, \phi) := E_{x \sim \mu_{ref}} [d(x, D_{\theta}(E_{\phi}(x)))]$. The optimal autoencoder for the given task (μ_{ref}, d) is then $\arg \min_{\theta, \phi} L(\theta, \phi)$.

ϕ }. The search for the optimal autoencoder can be accomplished by any mathematical optimization technique, but usually by gradient descent. This search process is referred to as "training the autoencoder". In most situations, the reference distribution is just the empirical distribution given by a dataset $\{x_1, \dots, x_N\} \subset \mathcal{X}$, so that
$$\mu_{ref} = \frac{1}{N} \sum_{i=1}^N \delta_{x_i}$$

-- 04 --

Information retrieval Information retrieval benefits particularly from dimensionality reduction in that search can become more efficient in certain kinds of low dimensional spaces. Autoencoders were indeed applied to semantic hashing, proposed by Salakhutdinov and Hinton in 2007. By training the algorithm to produce a low-dimensional binary code, all database entries could be stored in a hash table mapping binary code vectors to entries. This table would then support information retrieval by returning all entries with the same binary code as the query, or slightly less similar entries by flipping some bits from the query encoding.

-- 05 --

$L_{code} + L_{error}$, where L_{code} represents the length of the compressed latent representation and L_{error} denotes the reconstruction error.

-- 06 --

There exist representations to the messages that are relatively stable and robust to the type of noise we are likely to encounter; The said representations capture structures in the input distribution that are useful for our purposes. Example noise processes include:

-- 07 --

Machine translation Autoencoders have been applied to machine translation, which is usually referred to as neural machine translation (NMT). Unlike traditional autoencoders, the output does not match the input - it is in another language. In NMT, texts are treated as sequences to be encoded into the learning procedure, while on the decoder side sequences in the target language(s) are generated. Language-specific autoencoders incorporate further linguistic features into the learning procedure, such as Chinese decomposition features. Machine translation is rarely still done with autoencoders, due to the

availability of more effective transformer networks.

-- 08 --

An autoencoder is a type of artificial neural network used to learn efficient codings of unlabeled data (unsupervised learning). An autoencoder learns two functions: an encoding function that transforms the input data, and a decoding function that recreates the input data from the encoded representation. The autoencoder learns an efficient representation (encoding) for a set of data, typically for dimensionality reduction, to generate lower-dimensional embeddings for subsequent use by other machine learning algorithms. Variants exist which aim to make the learned representations assume useful properties. Examples are regularized autoencoders (sparse, denoising and contractive autoencoders), which are effective in learning representations for subsequent classification tasks, and variational autoencoders, which can be used as generative models. Autoencoders are applied to many problems, including facial recognition, feature detection, anomaly detection, and learning the meaning of words. In terms of data synthesis, autoencoders can also be used to randomly generate new data that is similar to the input (training) data.

-- 09 --

Interpretation An autoencoder has two main parts: an encoder that maps the message to a code, and a decoder that reconstructs the message from the code. An optimal autoencoder would perform as close to perfect reconstruction as possible, with "close to perfect" defined by the reconstruction quality function d . The simplest way to perform the copying task perfectly would be to duplicate the signal. To suppress this behavior, the code space Z usually has fewer dimensions than the message space X . Such an autoencoder is called undercomplete. It can be interpreted as compressing the message, or reducing its dimensionality. At the limit of an ideal undercomplete autoencoder, every possible code z in the code space is used to encode a message x that really appears in the distribution μ_{ref} , and the decoder is also perfect: $D_{\theta}(E_{\phi}(x)) = x$. This ideal autoencoder can then be used to generate messages indistinguishable from real messages, by feeding its decoder arbitrary code z and obtaining $D_{\theta}(z)$, which is a message that really appears in the distribution μ_{ref} . If the code space Z has dimension larger than (overcomplete), or equal to, the message space X , or the hidden units are given enough capacity, an

autoencoder can learn the identity function and become useless. However, experimental results found that overcomplete autoencoders might still learn useful features. In the ideal setting, the code dimension and the model capacity could be set on the basis of the complexity of the data distribution to be modeled. A standard way to do so is to add modifications to the basic autoencoder, to be detailed below.